

StackGAN++: Realistic Image Synthesis with Stacked Generative Adversarial Networks

Han Zhang^{ID}, Tao Xu^{ID}, Hongsheng Li, Shaoqing Zhang, *Senior Member, IEEE*, Xiaogang Wang, *Member, IEEE*, Xiaolei Huang, *Member, IEEE*, and Dimitris N. Metaxas, *Fellow, IEEE*

Abstract—Although Generative Adversarial Networks (GANs) have shown remarkable success in various tasks, they still face challenges in generating high quality images. In this paper, we propose Stacked Generative Adversarial Networks (StackGANs) aimed at generating high-resolution photo-realistic images. First, we propose a two-stage generative adversarial network architecture, StackGAN-v1, for text-to-image synthesis. The Stage-I GAN sketches the primitive shape and colors of a scene based on a given text description, yielding low-resolution images. The Stage-II GAN takes Stage-I results and the text description as inputs, and generates high-resolution images with photo-realistic details. Second, an advanced multi-stage generative adversarial network architecture, StackGAN-v2, is proposed for both conditional and unconditional generative tasks. Our StackGAN-v2 consists of multiple generators and multiple discriminators arranged in a tree-like structure; images at multiple scales corresponding to the same scene are generated from different branches of the tree. StackGAN-v2 shows more stable training behavior than StackGAN-v1 by jointly approximating multiple distributions. Extensive experiments demonstrate that the proposed stacked generative adversarial networks significantly outperform other state-of-the-art methods in generating photo-realistic images.

Index Terms—Generative models, generative adversarial networks (GANs), multi-stage GANs, multi-distribution approximation, photo-realistic image generation, text-to-image synthesis

1 INTRODUCTION

GENERATIVE Adversarial Networks (GANs) were proposed by Goodfellow et al. [12]. In the original setting, GANs are composed of a generator and a discriminator that are trained with competing goals. The generator is trained to generate samples towards the true data distribution to fool the discriminator, while the discriminator is optimized to distinguish between real samples from the true data distribution and fake samples produced by the generator. Recently, GANs have shown great potential in simulating complex data distributions, such as those of texts [6], images [32] and videos [48].

Despite the success, GANs are known to be difficult to train. The training process is usually unstable and sensitive to the choices of hyper-parameters. Several papers argued that the instability is partially due to the disjoint supports of

the data distribution and the implied model distribution [2], [42]. This problem is more severe when training GANs to generate high-resolution (e.g., 256×256) images because the chance is very low for the image distribution and the model distribution to share supports in a high-dimensional space. Moreover, a common failure phenomenon for GANs training is *mode collapse*, where many of the generated samples contain the same color or texture pattern.

In order to stabilize the GANs' training process and improve sample diversity, several methods tried to address the challenges by proposing new network architectures [32], introducing heuristic tricks [40] or modifying the learning objectives [1], [3], [5]. But most of the previous methods are designed to approximate the image distribution at a single scale. Due to the difficulty in directly approximating the high-resolution image data distribution, most previous methods are limited to generating low-resolution images. To circumvent this difficulty, we observe that, real world data, especially natural images, can be modeled at different scales [38]. One can view multi-resolution digitized images as samples from the same continuous image signal with different sampling rates. Henceforth, the distributions of images at multiple discrete scales are related. Apart from multiple distributions of different scales, images coupled with or without auxiliary conditioning variables (e.g., class labels or text descriptions) can be viewed as conditional distributions or unconditional distributions, which are also related distributions. Motivated by these observations, we argue that GANs can be stably trained to generate high resolution images by breaking the difficult generative task into sub-problems with progressive goals. Thus, we propose Stacked Generative

- H. Zhang and D. N. Metaxas are with the Department of Computer Science, Rutgers University, Piscataway, NJ 08854. E-mail: zhanghan8788@gmail.com, dnm@cs.rutgers.edu.
- T. Xu and X. Huang are with the Department of Computer Science and Engineering, Lehigh University, Bethlehem, PA 18015. E-mail: {tax313, xih206}@lehigh.edu.
- H. Li and X. Wang are with the Department of Electronic Engineering, The Chinese University of Hong Kong, Shatin, N. T., Hong Kong. E-mail: {hsli, xgwang}@ee.cuhk.edu.hk.
- S. Zhang is with the Department of Computer Science, University of North Carolina at Charlotte, Charlotte, NC 28223. E-mail: szhang16@uncc.edu.

Manuscript received 25 Dec. 2017; revised 14 May 2018; accepted 27 June 2018. Date of publication 15 July 2018; date of current version 11 July 2019. (Corresponding author: Dimitris N. Metaxas.)

Recommended for acceptance by A. Geiger.

For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below.

Digital Object Identifier no. 10.1109/TPAMI.2018.2856256

Adversarial Networks (StackGANs) to model a series of low-to-high-dimensional data distributions.

First, we propose a two-stage generative adversarial network, StackGAN-v1, to generate images from text descriptions through a sketch-refinement process [55]. Low-resolution images are first generated by our Stage-I GAN. On top of the Stage-I GAN, we stack Stage-II GAN to generate high-resolution (e.g., 256×256) images. By conditioning on the Stage-I result and the text again, Stage-II GAN learns to capture the text information that is omitted by Stage-I GAN and draws more details. Further, we propose a novel Conditioning Augmentation (CA) technique to encourage smoothness in the latent conditioning manifold [55]. It allows small random perturbations in the conditioning manifold and increases the diversity of synthesized images.

Second, we propose an advanced multi-stage generative adversarial network architecture, StackGAN-v2, for both conditional and unconditional generative tasks. StackGAN-v2 has multiple generators that share most of their parameters in a tree-like structure. As shown in Fig. 2, the input to the network can be viewed as the root of the tree, and multi-scale images are generated from different branches of the tree. The generator at the deepest branch has the final goal of generating photo-realistic high-resolution images. Generators at intermediate branches have progressive goals of generating small to large images to help accomplish the final goal. The whole network is jointly trained to approximate different but highly related image distributions at different branches. The positive feedback from modeling one distribution can improve the learning of others. For conditional image generation tasks, our proposed StackGAN-v2 simultaneously approximates the unconditional image-only distribution and the image distribution conditioned on text descriptions. Those two types of distributions are complementary to each other. Moreover, we propose a color-consistency regularization term to guide our generators to generate more coherent samples across different scales. The regularization provides additional constraints to facilitate multi-distribution approximation, which is especially useful in the unconditional setting where there is no instance-wise supervision between the image and the input noise vector.

In summary, the proposed Stacked Generative Adversarial Networks have three major contributions. (i) Our StackGAN-v1 for the first time generates images of 256×256 resolution with photo-realistic details from text descriptions. (ii) A new Conditioning Augmentation technique is proposed to stabilize the conditional GANs' training and also improve the diversity of the generated samples. (iii) Our StackGAN-v2 further improves the quality of generated images and stabilizes the GANs' training by jointly approximating multiple distributions. In the remainder of this paper, we first discuss related work and preliminaries in Sections 2 and 3, respectively. We then introduce our StackGAN-v1 [55] in Section 4 and StackGAN-v2 in Section 5. In Section 6, extensive experiments are conducted to evaluate the proposed methods. Finally, we make conclusions in Section 7. The source code for StackGAN-v1 is available at <https://github.com/hanzhanggit/StackGAN>, and the source code for StackGAN-v2 is available at <https://github.com/hanzhanggit/StackGAN-v2>.

2 RELATED WORK

Generative image modeling is a fundamental problem in computer vision. There has been remarkable progress in this direction with the emergence of deep learning techniques. Variational Autoencoders (VAEs) [21], [37] formulate the problem with probabilistic graphical models with the goal of maximizing the lower bound of data likelihood. Autoregressive models (e.g., PixelRNN) [44] that utilize neural networks to model the conditional distribution of the pixel space have also generated appealing synthetic images. Recently, Generative Adversarial Networks [12] have shown promising performance for generating sharper images. But the training instability makes it hard for GANs to generate high-resolution (e.g., 256×256) images. A lot of works have been proposed to stabilize the training and improve the image qualities [5], [27], [29], [32], [40], [56].

Built upon these generative models, conditional image generation has also been studied. Most methods utilize simple conditioning variables such as attributes or class labels [7], [31], [45], [52]. There are also works conditioned on images to generate images, including photo editing [4], [57], domain transfer [18], [43] and super-resolution [23], [42]. However, super-resolution methods [23], [42] can only add limited details to low-resolution images and can not correct large defects. In contrast, the latter stages in our proposed StackGANs can not only add details to low-resolution images generated by earlier stages but also correct defects in them. Recently, several methods have been developed to generate images from unstructured text. Mansimov et al. [25] built an AlignDRAW model by learning to estimate alignment between text and the generating canvas. Reed et al. [36] used conditional PixelCNN to generate images using text descriptions and object location constraints. Nguyen et al. [29] used an approximate Langevin sampling approach to generate images conditioned on text. However, their sampling approach requires an inefficient iterative optimization process. With conditional GANs, Reed et al. [35] successfully generated plausible 64×64 images for birds and flowers based on text descriptions. Their follow-up work [33] was able to generate 128×128 images by utilizing additional annotations on object part locations.

Given the difficulties in modeling details of natural images, many works have been proposed to use multiple GANs to improve sample quality. Wang et al. [50] utilized a structure GAN and a style GAN to synthesize images of indoor scenes. Yang et al. [53] factorized image generation into foreground and background generation with layered recursive GANs. Huang et al. [16] added several GANs to reconstruct the multi-level representations of a pre-trained discriminative model. But they were unable to generate high resolution images with photo-realistic details. Durugkar et al. [10] used multiple discriminators along with one generator to increase the chance of the generator receiving effective feedback. However, all discriminators in their framework are trained to approximate the image distribution at a single scale. Some methods [8], [19] follow the same intuition as our work. We all agree that it is beneficial to break the high-resolution image generation task into several easier subtasks to be accomplished in multiple stages. Denton et al. [8] built a series of GANs within a Laplacian pyramid framework (LAPGANs). At each level of the pyramid, a

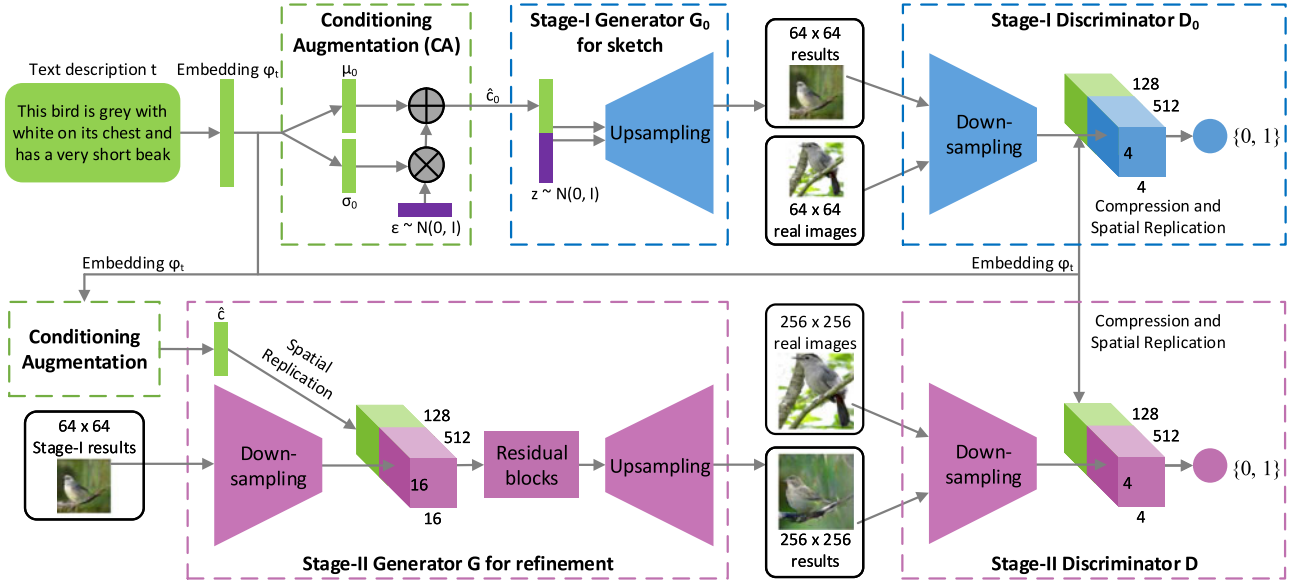


Fig. 1. The architecture of the proposed StackGAN-v1. The Stage-I generator draws a low-resolution image by sketching rough shape and basic colors of the object from the given text and painting the background from a random noise vector. Conditioned on Stage-I results, the Stage-II generator corrects defects and adds compelling details into Stage-I results, yielding a more realistic high-resolution image.

residual image conditioned on the image of the previous stage is generated and then added back to the input image to produce the input for the next stage. Instead of producing a residual image, our StackGANs directly generate high resolution images that are conditioned on their low-resolution inputs. Concurrent to our work, Kerras et al. [19] incrementally add more layers in the generator and discriminator for high resolution image generation. The main difference in terms of experimental setting is that they used a more restrained upsampling rule: starting from 4×4 pixels, their image resolution is increased by a factor of 2 between consecutive image generation stages. Furthermore, although StackGANs, LAPGANs [8] and Progressive GANs [19] all put emphasis on adding finer details in higher resolution images, our StackGANs can also correct incoherent artifacts or defects in low resolution results by utilizing an encoder-decoder network before the upsampling layers.

3 PRELIMINARIES

Generative Adversarial Networks [12] are composed of two models that are alternatively trained to compete with each other. The generator G is optimized to reproduce the true data distribution p_{data} by generating images that are difficult for the discriminator D to differentiate from real images. Meanwhile, D is optimized to distinguish real images and synthetic images generated by G . Overall, the training procedure is a minmax two-player game with the following objective function,

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))], \quad (1)$$

where x is a real image from the true data distribution p_{data} , and z is a noise vector sampled from the prior distribution p_z (e.g., uniform or Gaussian distribution). In practice, the generator G is modified to maximize $\log(D(G(z)))$ instead of minimizing $\log(1 - D(G(z)))$ to mitigate the problem

of gradient vanishing [12]. We use this modified non-saturating objective in all our experiments.

Conditional GANs [11], [28] are extension of GANs where both the generator and discriminator receive additional conditioning variables c , yielding $G(z, c)$ and $D(x, c)$. This formulation allows G to generate images conditioned on variables c .

4 STACKGAN-v1: TWO-STAGE GENERATIVE ADVERSARIAL NETWORK

To generate high-resolution images with photo-realistic details, we propose a simple yet effective two-stage generative adversarial network, StackGAN-v1. As shown in Fig. 1, it decomposes the text-to-image generative process into two stages. Stage-I GAN sketches the primitive shape and basic colors of the object conditioned on the given text description, and draws the background layout from a random noise vector, yielding a low-resolution image. Stage-II GAN corrects defects in the low-resolution image from Stage-I and completes details of the object by reading the text description again, producing a high-resolution photo-realistic image.

4.1 Conditioning Augmentation

As shown in Fig. 1, the text description t is first encoded by an encoder, yielding a text embedding ϕ_t . In previous works [33], [35], the text embedding is nonlinearly transformed to generate conditioning latent variables as the input of the generator. However, latent space for the text embedding is usually high dimensional (> 100 dimensions). With limited amount of data, it usually causes discontinuity in the latent data manifold, which is not desirable for learning the generator. To mitigate this problem, we introduce a Conditioning Augmentation technique to produce additional conditioning variables $\hat{c}$. In contrast to the fixed conditioning text variable c in [33], [35], we randomly sample the latent variables $\hat{c}$ from an independent Gaussian distribution $\mathcal{N}(\mu(\phi_t), \Sigma(\phi_t))$, where the mean

$\mu(\varphi_t)$ and diagonal covariance matrix $\Sigma(\varphi_t)$ are functions of the text embedding φ_t . The proposed Conditioning Augmentation yields more training pairs given a small number of image-text pairs, and thus encourages robustness to small perturbations along the conditioning manifold. To further enforce the smoothness over the conditioning manifold and avoid overfitting [9], [22], we add the following regularization term to the objective of the generator during training,

$$D_{KL}(\mathcal{N}(\mu(\varphi_t), \Sigma(\varphi_t)) \parallel \mathcal{N}(0, I)), \quad (2)$$

which is the Kullback-Leibler divergence (KL divergence) between the standard Gaussian distribution and the conditioning Gaussian distribution. The randomness introduced in the Conditioning Augmentation is beneficial for modeling text to image translation as the same sentence usually corresponds to objects with various poses and appearances.

4.2 Stage-I GAN

Instead of directly generating a high-resolution image conditioned on the text description, we simplify the task to first generate a low-resolution image with our Stage-I GAN, which focuses on drawing only rough shape and correct colors for the object.

Let φ_t be the text embedding of the given description. The Gaussian conditioning variables $\hat{c}_0$ for text embedding are sampled from $\mathcal{N}(\mu_0(\varphi_t), \Sigma_0(\varphi_t))$ to capture the meaning of φ_t with variations. Conditioned on $\hat{c}_0$ and random variable z , Stage-I GAN trains the discriminator D_0 and the generator G_0 by alternatively maximizing $\mathcal{L}_{D_0}$ in Eq. (3) and minimizing $\mathcal{L}_{G_0}$ in Eq. (4),

$$\mathcal{L}_{D_0} = \mathbb{E}_{(I_0, t) \sim p_{data}} [\log D_0(I_0, \varphi_t)] + \mathbb{E}_{z \sim p_z, t \sim p_{data}} [\log (1 - D_0(G_0(z, \hat{c}_0), \varphi_t))], \quad (3)$$

$$\mathcal{L}_{G_0} = \mathbb{E}_{z \sim p_z, t \sim p_{data}} [-\log D_0(G_0(z, \hat{c}_0), \varphi_t)] + \lambda D_{KL}(\mathcal{N}(\mu_0(\varphi_t), \Sigma_0(\varphi_t)) \parallel \mathcal{N}(0, I)), \quad (4)$$

where the real image I_0 and the text description t are from the true data distribution p_{data} . z is a noise vector randomly sampled from a given distribution p_z (Gaussian distribution in this paper). λ is a regularization parameter that balances the two terms in Eq. (4). We set $\lambda = 1$ for all our experiments. Using the reparameterization trick introduced in [21], both $\mu_0(\varphi_t)$ and $\Sigma_0(\varphi_t)$ are learned jointly with the rest of the network. To extract a visually-discriminative text embedding of the given description, we follow the approach of Reed et al. [34] to pre-train a text encoder. It is a character level CNN-RNN model that maps text descriptions to the common feature space of images by learning a correspondence function between texts with images [34].

Model Architecture. For the generator G_0 , to obtain text conditioning variable $\hat{c}_0$, the text embedding φ_t is first fed into a fully connected layer to generate μ_0 and σ_0 (σ_0 are the values in the diagonal of Σ_0) for the Gaussian distribution $\mathcal{N}(\mu_0(\varphi_t), \Sigma_0(\varphi_t))$. $\hat{c}_0$ are then sampled from the Gaussian distribution. Our N_g dimensional conditioning vector $\hat{c}_0$ is computed by $\hat{c}_0 = \mu_0 + \sigma_0 \odot \epsilon$ (where $\odot$ is the element-wise multiplication, $\epsilon \sim \mathcal{N}(0, I)$). Then, $\hat{c}_0$ is concatenated with a N_z dimensional noise vector to generate a $W_0 \times H_0$ image by a series of up-sampling blocks.

For the discriminator D_0 , the text embedding φ_t is first compressed to N_d dimensions using a fully-connected layer and then spatially replicated to form a $M_d \times M_d \times N_d$ tensor. Meanwhile, the image is fed through a series of down-sampling blocks until it has $M_d \times M_d$ spatial dimension. Then, the image filter map is concatenated along the channel dimension with the text tensor. The resulting tensor is further fed to a 1×1 convolutional layer to jointly learn features across the image and the text. Finally, a fully-connected layer with one node is used to produce the decision score.

4.3 Stage-II GAN

Low-resolution images generated by Stage-I GAN usually lack vivid object parts and might contain shape distortions. Some details in the text might also be omitted in the first stage, which is vital for generating photo-realistic images. Our Stage-II GAN is built upon Stage-I GAN results to generate high-resolution images. It is conditioned on low-resolution images and also the text embedding again to correct defects in Stage-I results. The Stage-II GAN completes previously ignored text information to generate more photo-realistic details.

Conditioning on the low-resolution result $s_0 = G_0(z, \hat{c}_0)$ and Gaussian latent variables $\hat{c}$, the discriminator D and generator G in Stage-II GAN are trained by alternatively maximizing $\mathcal{L}_D$ in Eq. (5) and minimizing $\mathcal{L}_G$ in Eq. (6),

$$\mathcal{L}_D = \mathbb{E}_{(I, t) \sim p_{data}} [\log D(I, \varphi_t)] + \mathbb{E}_{s_0 \sim p_{G_0}, t \sim p_{data}} [\log (1 - D(G(s_0, \hat{c}), \varphi_t))], \quad (5)$$

$$\mathcal{L}_G = \mathbb{E}_{s_0 \sim p_{G_0}, t \sim p_{data}} [-\log D(G(s_0, \hat{c}), \varphi_t)] + \lambda D_{KL}(\mathcal{N}(\mu(\varphi_t), \Sigma(\varphi_t)) \parallel \mathcal{N}(0, I)), \quad (6)$$

Different from the original formulation of GANs, the random noise z is not used in this stage with the assumption that the randomness has already been preserved by s_0 . Gaussian conditioning variables $\hat{c}$ used in this stage and $\hat{c}_0$ used in Stage-I GAN share the same pre-trained text encoder, generating the same text embedding φ_t . However, Stage-I and Stage-II Conditioning Augmentation have different fully connected layers for generating different means and standard deviations. In this way, Stage-II GAN learns to capture useful information in the text embedding that is omitted by Stage-I GAN.

Model Architecture. We design Stage-II generator as an encoder-decoder network with residual blocks [14]. Similar to the previous stage, the text embedding φ_t is used to generate the N_g dimensional text conditioning vector $\hat{c}$, which is spatially replicated to form a $M_g \times M_g \times N_g$ tensor. Meanwhile, the Stage-I result s_0 generated by Stage-I GAN is fed into several down-sampling blocks (i.e., encoder) until it has a spatial size of $M_g \times M_g$. The image features and the text features are concatenated along the channel dimension. The encoded image features coupled with text features are fed into several residual blocks, which are designed to learn multi-modal representations across image and text features. Finally, a series of up-sampling layers (i.e., decoder) are used to generate a $W \times H$ high-resolution image. Such a generator is able to help rectify defects in the input image while add more details to generate the realistic high-resolution image.

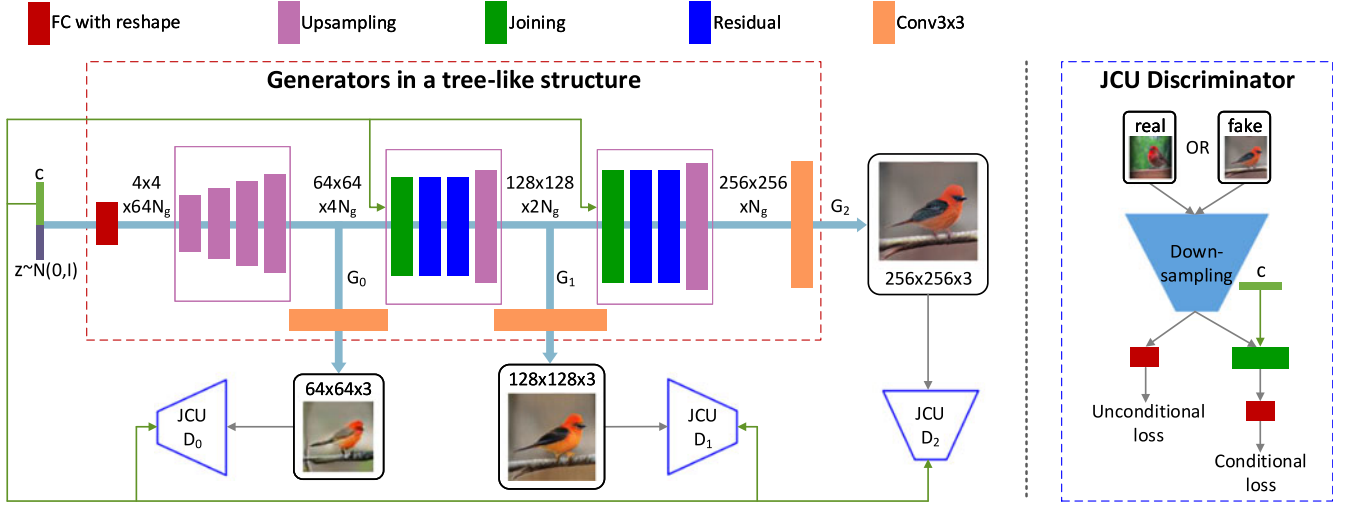


Fig. 2. The overall framework of our proposed StackGAN-v2 for the conditional image synthesis task. c is the vector of conditioning variables which can be computed from the class label, the text description, etc. N_g and N_d are the numbers of channels of a tensor.

For the discriminator, its structure is similar to that of Stage-I discriminator with only extra down-sampling blocks since the image size is larger in this stage. To explicitly enforce GANs to learn better alignment between the image and the conditioning text, rather than using the vanilla discriminator, we adopt the matching-aware discriminator proposed by Reed et al. [35] for both stages. During training, the discriminator takes real images and their corresponding text descriptions as positive sample pairs, whereas negative sample pairs consist of two groups. The first is real images with mismatched text embeddings, while the second is synthetic images with their corresponding text embeddings.

4.4 Implementation Details

The up-sampling blocks consist of the nearest-neighbor up-sampling followed by a 3×3 stride 1 convolution. Batch normalization [17] and ReLU activation are applied after every convolution except the last one. The residual blocks consist of 3×3 stride 1 convolutions, Batch normalization and ReLU. Two residual blocks are used in 128×128 StackGAN-v1 models while four are used in 256×256 models. The down-sampling blocks consist of 4×4 stride 2 convolutions, Batch normalization and LeakyReLU, except that the first one does not have Batch normalization.

By default, $N_g = 128$, $N_z = 100$, $M_g = 16$, $M_d = 4$, $N_d = 128$, $W_0 = H_0 = 64$ and $W = H = 256$. For training, we first iteratively train D_0 and G_0 of Stage-I GAN for 600 epochs by fixing Stage-II GAN. Then we iteratively train D and G of Stage-II GAN for another 600 epochs by fixing Stage-I GAN. All networks are trained using ADAM [20] solver with $\beta_1 = 0.5$. The batch size is 64. The learning rate is initialized to be 0.0002 and decayed to $1/2$ of its previous value every 100 epochs. The source code for StackGAN-v1 is available at <https://github.com/hanzhanggit/StackGAN> for more implementation details.

5 STACKGAN-V2: MULTI-DISTRIBUTION GENERATIVE ADVERSARIAL NETWORK

As discussed above, our StackGAN-v1 has two separate networks, Stage-I GAN and Stage-II GAN, to model low-to-high

resolution image distributions. To make the framework more general, in this paper, we propose a new end-to-end network, StackGAN-v2, to model a series of multi-scale image distributions. As shown in Fig. 2, StackGAN-v2 consists of multiple generators (G s) and discriminators (D s) in a tree-like structure. Images from low-resolution to high-resolution are generated from different branches of the tree. At each branch, the generator captures the image distribution at that scale and the discriminator estimates the probability that a sample came from training images of that scale rather than the generator.

The generators are jointly trained to approximate the multiple distributions, and the generators and discriminators are trained in an alternating fashion. In this section, we explore two types of multi-distributions: (1) multi-scale image distributions; and (2) joint conditional and unconditional image distributions.

5.1 Multi-Scale Image Distributions Approximation

Our StackGAN-v2 framework has a tree-like structure, which takes a noise vector $z \sim p_{noise}$ as the input and has multiple generators to produce images of different scales. The p_{noise} is a prior distribution, which is usually chosen as the standard normal distribution. The latent variables z are transformed to hidden features layer by layer. We compute the hidden features h_i for each generator G_i by a non-linear transformation,

$$h_0 = F_0(z); \quad h_i = F_i(h_{i-1}, z), i = 1, 2, \dots, m-1, \quad (7)$$

where h_i represents hidden features for the i th branch, m is the total number of branches, and F_i are modeled as neural networks (see Fig. 2 for illustration). In order to capture information omitted in preceding branches, the noise vector z is concatenated to the hidden features h_{i-1} as the inputs of F_i for calculating h_i . Based on hidden features at different layers ($h_0, h_1, \dots, h_{m-1}$), generators produce samples of small-to-large scales ($s_0, s_1, \dots, s_{m-1}$),

$$s_i = G_i(h_i), i = 0, 1, \dots, m-1, \quad (8)$$

where G_i is the generator for the i th branch.

Following each generator G_i , a discriminator D_i , which takes a real image x_i or a fake sample s_i as input, is trained

to classify inputs into two classes (real or fake) by minimizing the following cross-entropy loss,

$$\mathcal{L}_{D_i} = -\mathbb{E}_{x_i \sim p_{data_i}} [\log D_i(x_i)] - \mathbb{E}_{s_i \sim p_{G_i}} [\log (1 - D_i(s_i))], \quad (9)$$

where x_i is from the true image distribution p_{data_i} at the i th scale, and s_i is from the model distribution p_{G_i} at the same scale. The multiple discriminators are trained in parallel, and each of them focuses on a single image scale.

Guided by the trained discriminators, the generators are optimized to jointly approximate multi-scale image distributions ($p_{data_0}, p_{data_1}, \dots, p_{data_{m-1}}$) by minimizing the following loss function,

$$\mathcal{L}_G = \sum_{i=0}^{m-1} \mathcal{L}_{G_i}, \quad \mathcal{L}_{G_i} = -\mathbb{E}_{s_i \sim p_{G_i}} [\log D_i(s_i)], \quad (10)$$

where $\mathcal{L}_{G_i}$ is the loss function for approximating the image distribution at the i th scale (i.e., p_{data_i}). During the training process, the discriminators D_i and the generators G_i are alternately optimized till convergence.

The motivation of the proposed StackGAN-v2 is that, by modeling data distributions at multiple scales, if any one of those model distributions shares support with the real data distribution at that scale, the overlap could provide good gradient signal to expedite or stabilize training of the whole network at multiple scales. For instance, approximating the low-resolution image distribution at the first branch results in images with basic color and structures. Then the generators at the subsequent branches can focus on completing details for generating higher resolution images.

5.2 Joint Conditional and Unconditional Distribution Approximation

For unconditional image generation, discriminators in StackGAN-v2 are trained to distinguish real images from fake ones. To handle conditional image generation, conventionally, images and their corresponding conditioning variables are input into the discriminator to determine whether an image-condition pair matches or not, which guides the generator to approximate the conditional image distribution. We propose conditional StackGAN-v2 that jointly approximates conditional and unconditional image distributions.

For the generator of our conditional StackGAN-v2, F_0 and F_i are converted to take the conditioning vector c as input, such that $h_0 = F_0(c, z)$ and $h_i = F_i(h_{i-1}, c)$. For F_i , the conditioning vector c replaces the noise vector z to encourage the generators to draw images with more details according to the conditioning variables. Consequently, multi-scale samples are now generated by $s_i = G_i(h_i)$. The objective function of training the discriminator D_i for conditional StackGAN-v2 now consists of two terms, the unconditional loss and the conditional loss,

$$\begin{aligned} \mathcal{L}_{D_i} = & \underbrace{-\mathbb{E}_{x_i \sim p_{data_i}} [\log D_i(x_i)] - \mathbb{E}_{s_i \sim p_{G_i}} [\log (1 - D_i(s_i))]}_{\text{unconditional loss}} \\ & + \underbrace{-\mathbb{E}_{x_i \sim p_{data_i}} [\log D_i(x_i, c)] - \mathbb{E}_{s_i \sim p_{G_i}} [\log (1 - D_i(s_i, c))]}_{\text{conditional loss}}. \end{aligned} \quad (11)$$

The unconditional loss determines whether the image is real or fake while the conditional one determines whether the image and the condition match or not. Accordingly, the loss function for each generator G_i is converted to

$$\mathcal{L}_{G_i} = \underbrace{-\mathbb{E}_{s_i \sim p_{G_i}} [\log D_i(s_i)]}_{\text{unconditional loss}} + \underbrace{-\mathbb{E}_{s_i \sim p_{G_i}} [\log D_i(s_i, c)]}_{\text{conditional loss}}. \quad (12)$$

The generator G_i at each scale therefore jointly approximates unconditional and conditional image distributions. The final loss for jointly training generators of conditional StackGAN-v2 is computed by substituting Eq. (12) into Eq. (10).

5.3 Color-Consistency Regularization

As we increase the image resolution at different generators, the generated images at different scales should share similar basic structure and colors. A color-consistency regularization term is introduced to keep samples generated from the same input at different generators more consistent in color and thus to improve the quality of the generated images.

Let $\mathbf{x}_k = (R, G, B)^T$ represent a pixel in a generated image, then the mean and covariance of pixels of the given image can be defined by $\boldsymbol{\mu} = \sum_k \mathbf{x}_k / N$ and $\boldsymbol{\Sigma} = \sum_k (\mathbf{x}_k - \boldsymbol{\mu})(\mathbf{x}_k - \boldsymbol{\mu})^T / N$, where N is the number of pixels in the image. The color-consistency regularization term aims at minimizing the differences of $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ between different scales to encourage the consistency, which is defined as

$$\mathcal{L}_{C_i} = \frac{1}{n} \sum_{j=1}^n \left(\lambda_1 \|\boldsymbol{\mu}_{s_i^j} - \boldsymbol{\mu}_{s_{i-1}^j}\|_2^2 + \lambda_2 \|\boldsymbol{\Sigma}_{s_i^j} - \boldsymbol{\Sigma}_{s_{i-1}^j}\|_F^2 \right), \quad (13)$$

where n is the batch size, $\boldsymbol{\mu}_{s_i^j}$ and $\boldsymbol{\Sigma}_{s_i^j}$ are mean and covariance for the j th sample generated by the i th generator. Empirically, we set $\lambda_1 = 1$ and $\lambda_2 = 5$ by default. For the j th input vector, multi-scale samples $s_0^j, s_1^j, \dots, s_{m-1}^j$ are generated from m generators of StackGAN-v2. $\mathcal{L}_{C_i}$ can be added to the loss function of the i th generator defined in Eqs. (10) or (12), where $i = 1, 2, \dots, m-1$. Therefore, the final loss for training the i th generator is defined as $\mathcal{L}'_{G_i} = \mathcal{L}_{G_i} + \alpha \cdot \mathcal{L}_{C_i}$. Experimental results indicate that the color-consistency regularization is very important (e.g., $\alpha = 50.0$ in this paper) for the unconditional task, while it is not needed ($\alpha = 0.0$) for the text-to-image synthesis task which has a stronger constraint, i.e., the instance-wise correspondence between images and text descriptions.

5.4 Implementation Details

As shown in Fig. 2, our StackGAN-v2 models are designed to generate 256×256 images. The input vector (i.e., z for unconditional StackGAN-v2, or the concatenated z and c^1 for conditional StackGAN-v2) is first transformed to a $4 \times 4 \times 64N_g$ feature tensor, where N_g is the number of channels in the tensor. Then, this $4 \times 4 \times 64N_g$ tensor is gradually transformed to $64 \times 64 \times 4N_g$, $128 \times 128 \times 2N_g$, and eventually $256 \times 256 \times 1N_g$ tensors at different layers of the network by six up-sampling blocks. The intermediate $64 \times 64 \times 4N_g$, $128 \times 128 \times 2N_g$, and $256 \times 256 \times 1N_g$ features are used to generate images of corresponding scales with 3×3 convolutions.

1. The conditioning variable c for StackGAN-v2 is also generated by Conditioning Augmentation.

TABLE 1
Statistics of Datasets

Dataset	CUB [49]		Oxford-102 [30]		COCO [24]		LSUN [54]		ImageNet [39]	
	train	test	train	test	train	test	bedroom	church	dog	cat
#Samples	8,855	2,933	7,034	1,155	80,000	40,000	3,033,042	126,227	147,873	6,500

We do not split LSUN or ImageNet because they are utilized for the unconditional tasks.

Conditioning variables c or unconditional variables z are also directly fed into intermediate layers of the network to ensure encoded information in c or z is not omitted. All the discriminators D_i have down-sampling blocks and 3×3 convolutions to transform the input image to a $4 \times 4 \times 8N_d$ tensor, and eventually the sigmoid function is used for outputting probabilities. For all datasets, we set $N_g = 32$, $N_d = 64$ and use two residual blocks between every two generators. ADAM [20] solver with $\beta_1 = 0.5$ and a learning rate of 0.0002 is used for all models. The source code for StackGAN-v2 is available at <https://github.com/hanzhanggit/StackGAN-v2> for more implementation details.

6 EXPERIMENTS

We conduct extensive experiments to evaluate the proposed methods. In Section 6.1, several state-of-the-art methods on text-to-image synthesis and on unconditional image synthesis are compared with the proposed methods. We first evaluate the effectiveness of our StackGAN-v1 for text-to-image synthesis by comparing it with GAWWN [33] and GAN-INT-CLS [35]. And then, StackGAN-v2 is compared with StackGAN-v1 on different datasets to show its advantages and limitations. Moreover, StackGAN-v2 as a more general framework also works well on unconditional image synthesis tasks, and on such tasks, it is compared with several state-of-the-art methods [3], [13], [26], [32], [56]. In Section 6.2, several baseline models are designed to investigate the overall design and important components of our StackGAN-v1. For the first baseline, we directly train Stage-I GAN for generating 64×64 and 256×256 images to investigate whether the proposed two-stage stacked structure and the Conditioning Augmentation are beneficial. Then we modify our StackGAN-v1 to generate 128×128 and 256×256 images to investigate whether larger images by our method can result in higher image quality. We also investigate whether inputting text at both stages of StackGAN-v1 is useful. In Section 6.3, experiments are designed to validate important components of our StackGAN-v2, including designs with fewer multi-scale image distributions, the effect of jointly approximating conditional and unconditional distributions, and the effectiveness of the proposed color-consistency regularization.

Datasets. We evaluate our conditional StackGAN for text-to-image synthesis on the CUB [49], Oxford-102 [30] and COCO [24] datasets. CUB [49] contains 200 bird species with 11,788 images. Since 80 percent of birds in this dataset have object-image size ratios of less than 0.5 [49], as a pre-processing step, we crop all images to ensure that bounding boxes of birds have greater-than-0.75 object-image size ratios. Oxford-102 [30] contains 8,189 images of flowers from 102 different categories. To show the generalization capability of our approach, a more challenging dataset, COCO [24] is also

utilized for evaluation. Different from CUB and Oxford-102, the COCO dataset contains images with multiple objects and various backgrounds. Each image in COCO has 5 descriptions, while 10 descriptions are provided by [34] for every image in CUB and Oxford-102 datasets. Following the experimental setup in [35], we directly use the training and validation sets provided by COCO, meanwhile we split CUB and Oxford-102 into class-disjoint training and test sets. Our unconditional StackGAN utilizes bedroom and church sub-sets of LSUN [54], a dog-breed² and a cat-breed³ sub-sets of ImageNet [39] to synthesize different types of images. The statistics of datasets are presented in Table 1.

Evaluation Metrics. It is difficult to evaluate the performance of generative models (e.g., GANs). In this paper, we choose inception score (IS) [40] and fr chet inception distance (FID) [15] for quantitative evaluation. Inception score (IS) [40] is the first well-known metric for evaluating GANs. $IS = \exp(\mathbb{E}_x D_{KL}(p(y|x) || p(y)))$, where x denotes one generated sample, and y is the label predicted by the inception model [41]⁴. The intuition behind this metric is that good models should generate diverse but meaningful images. Therefore, the KL divergence between the marginal distribution $p(y)$ and the conditional distribution $p(y|x)$ should be large. As suggested in [40], we compute the inception score on a large number of samples (i.e., 30k samples randomly generated for the test set) for each model.⁵

Fr chet inception distance [15] was recently proposed as a metric that considers not only the synthetic data distribution but also how it compares to the real data distribution. It directly measures the distance between the synthetic data distribution $p(\cdot)$ and the real data distribution $p_r(\cdot)$. In practice, images are encoded with visual features by the inception model. Assuming the feature embeddings follow a multidimensional Gaussian distribution, the synthetic data's Gaussian with mean and covariance (m, C) is obtained from $p(\cdot)$ and the real data's Gaussian with mean and covariance (m_r, C_r) is obtained from $p_r(\cdot)$. The difference between the synthetic and real Gaussians is measured by the Fr chet distance, i.e., $FID = ||m - m_r||_2^2 + Tr(C + C_r - 2(CC_r)^{1/2})$. Lower FID values mean closer distances between synthetic and real data distributions. To compute the FID score for a unconditional model, 30k samples are randomly generated. To compute the FID score for a text-to-image model, all

2. Using the wordNet IDs provided by Vinyals et al., [47].

3. Using the wordNet IDs provided in our supplementary materials, which can be found on the Computer Society Digital Library at <http://doi.ieeecomputersociety.org/10.1109/TPAMI.2018.2856256>

4. In our experiments, for fine-grained datasets, CUB and Oxford-102, we fine-tune an inception model for each of them. For other datasets, we directly use the pre-trained inception model.

5. The mean and standard derivation inception scores of ten splits are reported.

TABLE 2
Inception Scores (IS), Fréchet Inception Distance (FID) and Average Human Ranks (HR) of GAN-INT-CLS [35], GAWWN [33] and Our StackGAN-v1 on CUB, Oxford-102, and COCO

Metric	CUB			Oxford		COCO	
	GAN-INT-CLS	GAWWN	Our StackGAN-v1	GAN-INT-CLS	Our StackGAN-v1	GAN-INT-CLS	Our StackGAN-v1
FID ↓	68.79	67.22	51.89	79.55	55.28	60.62	74.05
FID* ↓	68.79	53.51	35.11	79.55	43.02	60.62	33.88
IS ↑	2.88 ± .04	3.62 ± .07	3.70 ± .04	2.66 ± .03	3.20 ± .01	7.88 ± .07	8.45 ± .03
IS* ↑	2.88 ± .04	3.10 ± .03	3.02 ± .03	2.66 ± .03	2.73 ± .03	7.88 ± .07	8.35 ± .11
HR ↓	2.76 ± .01	1.95 ± .02	1.29 ± .02	1.84 ± .02	1.16 ± .02	1.82 ± .03	1.18 ± .03

(* means that images are re-sized to 64×64 before computing IS* and FID*).

sentences in the corresponding test set are utilized to generate samples.

To better evaluate the proposed methods, especially to see whether the generated images are well conditioned on the given text descriptions, we also conduct user studies. We randomly select 50 text descriptions for each class of CUB and Oxford-102 test sets. For COCO dataset, 4k text descriptions are randomly selected from its validation set. For each sentence, 5 images are generated by each model. Given the same text descriptions, 30 users (not including any of the authors) are asked to rank the results by different methods. The average ranks by human users are calculated to evaluate all compared methods.

In addition, we use t-SNE [46] embedding method to visualize a large number (e.g., 30k on the CUB test set) of high-dimensional images in a two-dimensional map. We observe that t-SNE is a good tool to examine the distribution of synthesized images and identify collapsed modes.

6.1 Comparison with State-of-the-Art GAN Models

To demonstrate the effectiveness of the proposed method, we compare it with state-of-the-art GAN models on text-to-image synthesis [33], [35] and unconditional image synthesis [3], [13], [26], [32], [56].

Text-to-image synthesis. We compare our StackGAN models with several state-of-the-art text-to-image methods [33], [35] on CUB, Oxford-102 and COCO datasets. The inception scores, fr chet inception distances and average human ranks for the proposed StackGAN models and compared methods are reported in Table 2. Representative examples are compared in Figs. 3, 4. For meaningful and fair comparisons with previous methods, the inception scores (IS/IS*) and fr chet inception distances (FID/FID*) are computed in two settings. In the first setting, 256×256 images produced by StackGAN, 128×128 images generated by GAWWN [33] and 64×64 images yielded by GAN-INT-CLS [35] are used directly to compute IS and FID. Thus, in this setting, the different



Fig. 3. Example results by our StackGANs, GAWWN [33], and GAN-INT-CLS [35] conditioned on text descriptions from CUB test set.

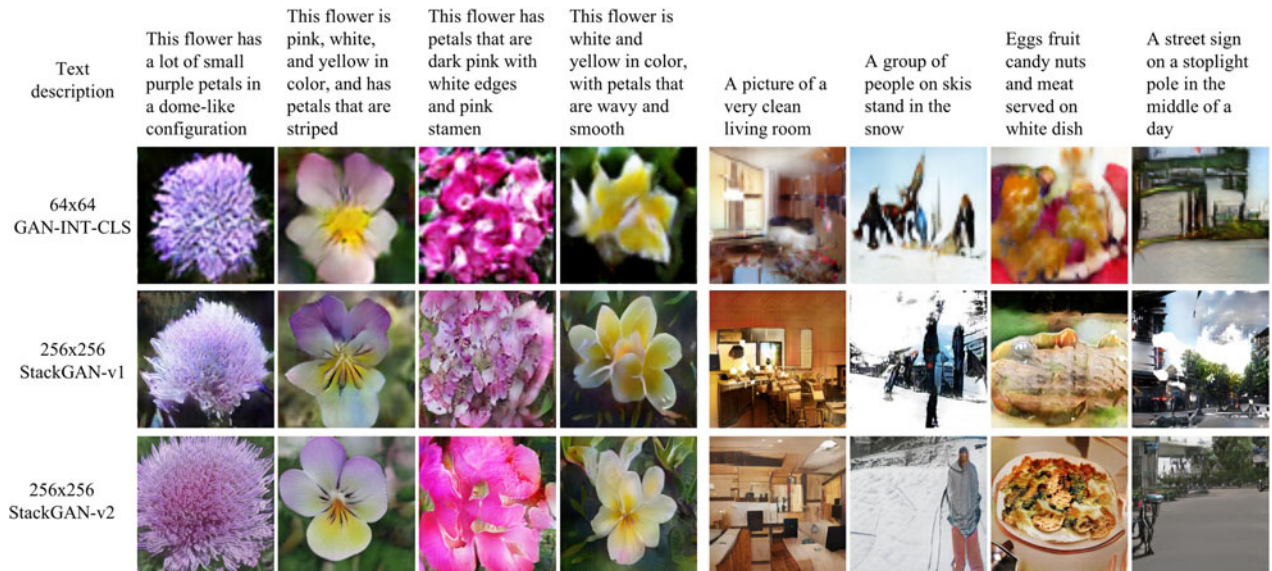


Fig. 4. Example results by our StackGANs and GAN-INT-CLS [35] conditioned on text descriptions from Oxford-102 test set (leftmost four columns) and COCO validation set (rightmost four columns).

models are compared directly using their generated images, which have different resolutions. In the second setting, before computing IS* and FID*, all generated images are re-sized to the same resolution of 64×64 for fair comparison.

Compared with previous GAN models [33], [35], on the text-to-image synthesis task, our StackGAN-v1 model achieves the best FID*, IS and average human rank on all three datasets. As shown in Table 2, compared with GAN-INT-CLS [35], StackGAN-v1 achieves 28.47 percent improvement in terms of inception score (IS) on CUB dataset (from 2.88 to 3.70), and 20.30 percent improvement on Oxford-102 (from 2.66 to 3.20). When we compare images of different models at the same resolution of 64×64 , our StackGAN-v1 still achieves higher inception scores (IS*) than GAN-INT-CLS, but produces a slightly worse inception score (IS*) than GAWWN [33] because GAWWN uses additional supervision. Meanwhile, the FID* of StackGAN-v1 is nearly one half of the FID* of GAN-INT-CLS on each dataset. It means that the StackGAN-v1 can better model and estimate the 64×64 image distribution. As comparison, the FID of StackGAN-v1 is higher than that of GAN-INT-CLS [35] on COCO. The reason is that the FID of GAN-INT-CLS is the distance between two 64×64 image distributions while the FID of StackGAN-v1 is the distance between two 256×256 image distributions. It is clear that estimating the 64×64 image distribution is much easier than estimating the 256×256 image distribution. It is also the reason why the FID is higher than the FID* for the same model. Finally, the better average human rank of our StackGAN-v1 also indicates our proposed method is able to generate more realistic samples conditioned on text descriptions. On the other hand, representative examples are shown in Figs. 3 and 4 for visualization comparison. As shown in Fig. 3, the 64×64 samples generated by GAN-INT-CLS [35] can only reflect the general shape and color of the birds. Their results lack vivid parts (e.g., beak and legs) and convincing details in most cases, which make them neither realistic enough nor have sufficiently high resolution. By using additional conditioning variables on location constraints, GAWWN [33] obtains a better inception score on

CUB dataset, which is still slightly lower than ours. It generates higher resolution images with more details than GAN-INT-CLS, as shown in Fig. 3. However, as mentioned by its authors, GAWWN fails to generate any plausible images when it is only conditioned on text descriptions [33]. In comparison, our StackGAN-v1 for the first time generates images of 256×256 resolution with photo-realistic details from only text descriptions.

Comparison between StackGAN-v1 and StackGAN-v2. The comparison between StackGAN-v1 and StackGAN-v2 by different quantitative metrics as well as human evaluations are reported in Table 3. For unconditional generation, the samples generated by StackGAN-v2 are consistently better than those by StackGAN-v1 (last four columns in Table 3) from a human perspective. The end-to-end training scheme together with the color-consistency regularization enables StackGAN-v2 to produce more feedback and regularization for each branch so that consistency is better maintained during the multi-step generation process. This is especially useful for unconditional generation as no extra conditions (e.g., text) are applied. On the text-to-image datasets, the scores are mixed for StackGAN-v1 and StackGAN-v2. The reason is partially due to the fact that the text information, which is a strong constraint, is added in all the stages to keep coherence. The comparison results of FIDs are consistent with the comparison results of human ranks on all datasets. On the other hand, the inception score draws different conclusions on LSUN-bedroom, LSUN-church, and ImageNet-cat. We think that the reason is because the inception model is pre-trained on ImageNet with 1000 classes, which makes it less suitable for class-specific datasets. Compared with ImageNet-cat which has 17 classes, the inception score for ImageNet-dog correlates better with human ranks because ImageNet-dog covers more (i.e., 118) classes from ImageNet. Hence we believe that, using class-specific datasets, it is more reasonable to use FID to directly compare feature distances between generated samples with that of the real world samples [15].

For visual comparison of the results by the two models, we utilize the t-SNE [46] algorithm. For each model, a large

TABLE 3
Comparison of StackGAN-v1 and StackGAN-v2 on Different Datasets by Inception Scores (IS),
Fréchet Inception Distance (FID) and Average Human Ranks (HR)

Dataset		CUB	Oxford-102	COCO	LSUN-bedroom	LSUN-church	ImageNet-dog	ImageNet-cat
FID ↓	StackGAN-v1	51.89	55.28	74.05	91.94	57.20	89.21	58.73
	StackGAN-v2	15.30	48.68	81.59	35.61	25.36	44.54	28.59
IS ↑	StackGAN-v1	3.70 ± .04	3.20 ± .01	8.45 ± .03	3.59 ± .05	2.87 ± .05	8.84 ± .08	4.77 ± .06
	StackGAN-v2	4.04 ± .05	3.26 ± .01	8.30 ± .10	3.02 ± .04	2.38 ± .03	9.55 ± .11	4.23 ± .05
HR ↓	StackGAN-v1	1.81 ± .02	1.70 ± .03	1.45 ± .04	1.95 ± .01	1.86 ± .02	1.90 ± .01	1.88 ± .02
	StackGAN-v2	1.19 ± .02	1.30 ± .03	1.55 ± .05	1.05 ± .01	1.14 ± .02	1.10 ± .01	1.12 ± .02

number of images are generated and embedded into the 2D plane. We first extract a 2048d CNN feature from each generated image using a pre-trained Inception model. Then, t-SNE algorithm is applied to embed the CNN features into a 2D plane, resulting a location for each image in the 2D plane. Due to page limits, Fig. 5 only shows a 50×50 grid with compressed images for each dataset, where each generated image is mapped to its nearest grid location. By visualizing a large number of images, the t-SNE is a good tool to examine the synthesized distribution and evaluate its diversity. We also follow [31] to use the multiscale structural similarity (MS-SSIM) [51] as a metric to measure the variability of samples. We observe that the MS-SSIM is useful to find large-scale mode collapses but often fails to detect small-scale mode collapses or fails to measure the loss of variation in the generated samples' color or texture. This observation is consistent with the one found in [19]. For example, in Fig. 5, StackGAN-v1 has two small collapsed modes (nonsensical images) while StackGAN-v2 does not have any collapsed nonsensical mode. However, the MS-SSIM score of StackGAN-v1 (0.0945) is better than that of StackGAN-v2 (0.1311) and even better than that of the real data (0.1007). Thus, we argue that the MS-SSIM is not a good metric to capture small-scale mode collapses. On the contrary, the t-

SNE visualization of the generated samples can easily help us identify any collapsed modes in the samples as well as evaluate sample variability in texture, color and viewpoint.

More visual comparison of StackGAN-v1 and StackGAN-v2 on different datasets can be found in Figs 3, 4, 6, 7, 8, and 9. Specially, Fig. 9 illustrates failure cases of StackGAN-v1 and StackGAN-v2. We categorize the failures in these cases into three groups: mild, moderate, and severe. The "mild" group means that the generated images have smooth and coherent appearance but lack vivid objects. The "moderate" group means that the generated images have obvious artifacts, which usually are signs of mode collapse. The "severe" group indicates that the generated images fall into collapsed modes. Based on such criterion, on the simple dataset, Oxford-102, all failure cases of StackGAN-v1 belong to the "mild" group, while on other datasets all three groups of failure cases are observed. As comparison, we observe that all failure cases of StackGAN-v2 belong to the "mild" group, meaning StackGAN-v2-generated images have no collapsed nonsensical mode (see Fig. 5). By jointly optimizing multiple distributions (objectives), StackGAN-v2 shows more stable training behavior and results in better FID and inception scores on most datasets (see Table 3). However, because of the same reason, compared with StackGAN-v1,



(a) StackGAN-v1 has two collapsed modes (in red rectangles). (b) StackGAN-v2 contains no collapsed nonsensical mode.

Fig. 5. Utilizing t-SNE to embed images generated by our StackGAN-v1 and StackGAN-v2 on the CUB test set.

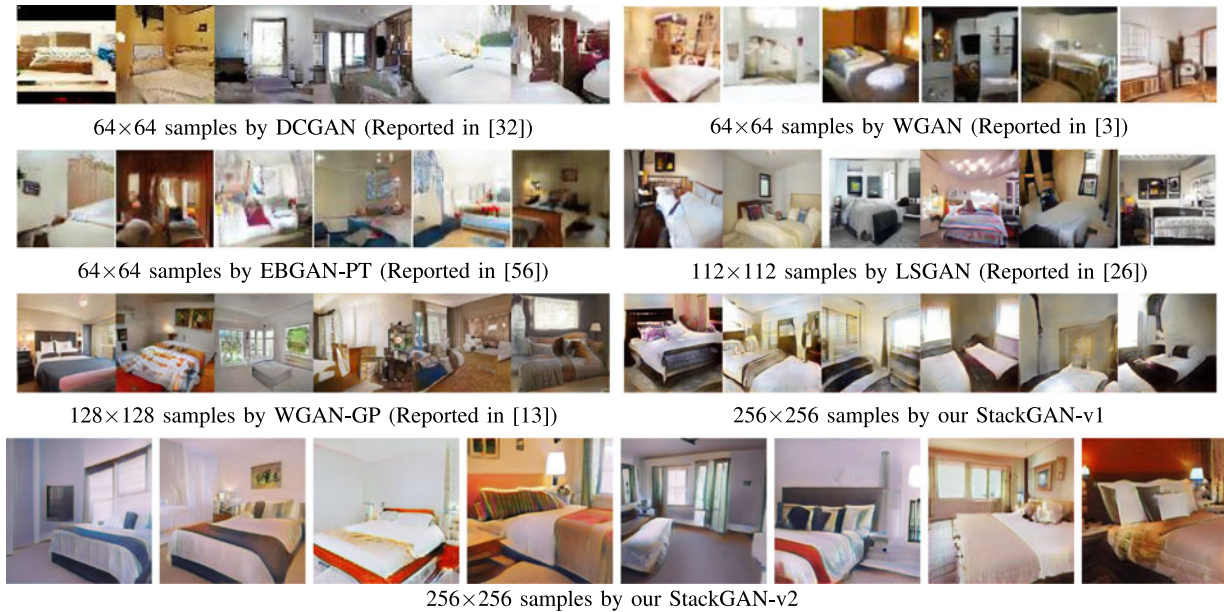


Fig. 6. Comparison of samples generated by models trained on LSUN bedroom dataset (Zoom in for better comparison).

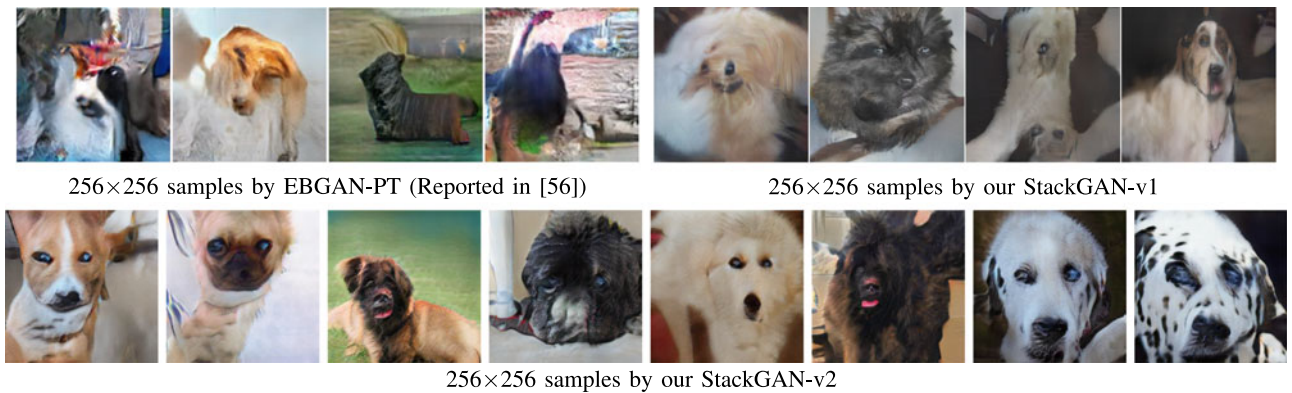


Fig. 7. Comparison of samples generated by models trained on ImageNet dog dataset.

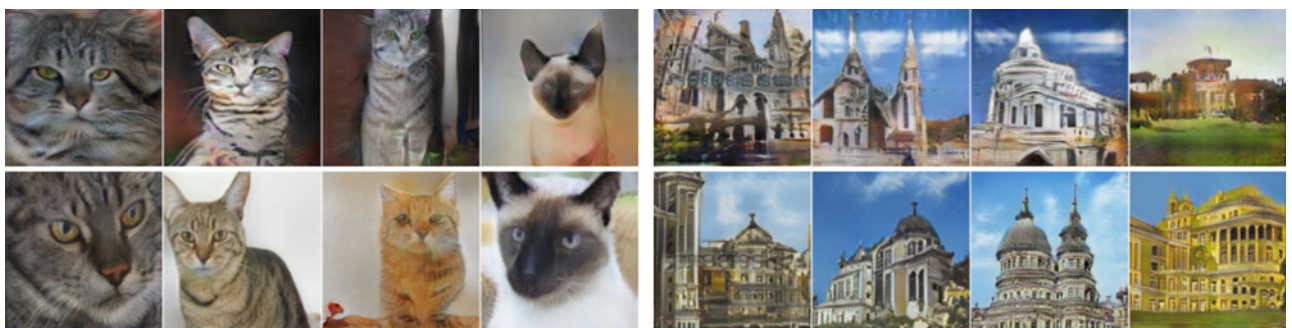


Fig. 8. 256×256 samples generated by our StackGAN-v1 (top) and StackGAN-v2 (bottom) on ImageNet cat (left) and LSUN church (right).

it is harder for StackGAN-v2 to converge on more complex datasets, such as COCO. In contrast, StackGAN-v1 optimizes sub-tasks separately by training stage by stage. It produces slightly more appealing images on COCO than StackGAN-v2 based on human rank results, but also generates more images that are moderate or severe failure cases. Consequently, while StackGAN-v2 is more advanced than StackGAN-v1 in many aspects (such as end-to-end training and more stable training behavior), StackGAN-v1 has the advantage of stage-by-stage training, which converges faster and requires less GPU memory.

Unconditional Image Synthesis. We evaluate the effectiveness of StackGAN-v2 for the unconditional image generation task by comparing it with DCGAN [32], WGAN [3], EBGAN-PT [56], LSGAN [26], and WGAN-GP [13] on the LSUN bedroom dataset. As shown in Fig. 6, our StackGAN-v2 is able to generate 256×256 images with more photo-realistic details. In Fig. 7, we also compare the 256×256 samples generated by StackGAN-v2 and EBGAN-PT. As shown in the figure, the samples generated by the two methods have the same resolution, but StackGAN-v2 generates more realistic ones (e.g., more recognizable dog faces



Fig. 9. Examples of failure cases of StackGAN-v1 (top) and StackGAN-v2 (bottom) on different datasets.



Fig. 10. Samples generated by our StackGAN-v1 from unseen texts in CUB test set. Each column lists the text description, images generated from the text by Stage-I and Stage-II of StackGAN-v1.

with eyes and noses). While on LSUN bedroom dataset, only qualitative results are reported in [3], [13], [26], [32], [56], a DCGAN model [32] is trained for quantitative comparison using the public available source code⁶ on the ImageNet Dog dataset. The inception score of DCGAN is 8.19 ± 0.11 which is much lower than the inception achieved by our StackGAN-v2 (9.55 ± 0.11). These experiments demonstrate that our StackGAN-v2 outperforms the state-of-the-art methods for unconditional image generation. Example images generated by StackGAN on LSUN church and ImageNet cat datasets are presented in Fig. 8.

6.2 The Component Analysis of StackGAN-v1

In this section, we analyze different components of StackGAN-v1 on CUB dataset with baseline models.

The Design of StackGAN-v1. As shown in the first four rows of Table 4, if Stage-I GAN is directly used to generate images, the inception scores decrease significantly. Such performance drop can be well illustrated by results in Fig. 11. As shown in the first row of Fig. 11, Stage-I GAN fails to generate any plausible 256×256 samples without using Conditioning Augmentation (CA). Although Stage-I GAN with CA is able to generate more diverse 256×256 samples, those samples are not as realistic as samples generated by StackGAN-v1. It demonstrates the necessity of the proposed stacked structure. In addition, by decreasing the output resolution from 256×256 to 128×128 , the inception score decreases from 3.70 to 3.35. Note that all images are scaled to 299×299 before calculating the inception score. Thus, if our StackGAN-v1 just increases the image size

without adding more information, the inception score would remain the same for samples of different resolutions. Therefore, the decrease in inception score by 128×128 StackGAN-v1 demonstrates that our 256×256 StackGAN-v1 does add more details into the larger images. For the 256×256 StackGAN-v1, if the text is only input to Stage-I (denoted as “no Text twice”), the inception score decreases from 3.70 to 3.45. It indicates that processing text descriptions again at Stage-II helps refine Stage-I results. The same conclusion can be drawn from the results of 128×128 StackGAN-v1 models.

Fig. 10 illustrates some examples of the Stage-I and Stage-II images generated by our StackGAN-v1. As shown in the first row of Fig. 10, in most cases, Stage-I GAN is able to draw rough shapes and colors of objects given text

TABLE 4
Inception Scores Calculated with 30,000 Samples Generated on CUB by Different Baseline Models of Our StackGAN-v1

Method	CA	Text twice	Inception score
64×64 Stage-I GAN	no	/	$2.66 \pm .03$
	yes	/	$2.95 \pm .02$
256×256 Stage-I GAN	no	/	$2.48 \pm .00$
	yes	/	$3.02 \pm .01$
128×128 StackGAN-v1	yes	no	$3.13 \pm .03$
	no	yes	$3.20 \pm .03$
	yes	yes	$3.35 \pm .02$
256×256 StackGAN-v1	yes	no	$3.45 \pm .02$
	no	yes	$3.31 \pm .03$
	yes	yes	$3.70 \pm .04$

6. <https://github.com/carpedm20/DCGAN-tensorflow>

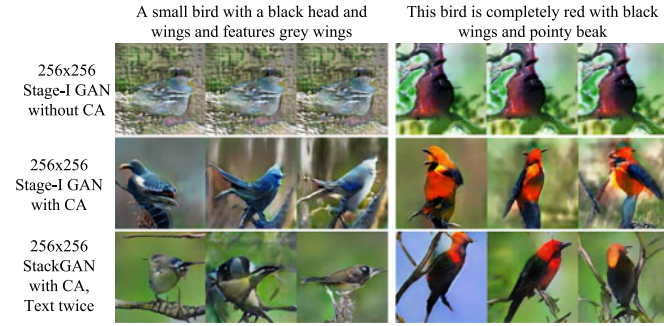


Fig. 11. Conditioning Augmentation (CA) helps stabilize the training of conditional GAN and improves the diversity of the generated samples. (Row 1) without CA, Stage-I GAN fails to generate plausible 256×256 samples. Although different noise vector z is used for each column, the generated samples collapse to be the same for each input text description. (Row 2-3) with CA but fixing the noise vectors z , methods are still able to generate birds with different poses and viewpoints.

descriptions. However, Stage-I images are usually blurry with various defects and missing details, especially for foreground objects. As shown in the second row, Stage-II GAN generates $4 \times$ higher resolution images with more convincing details to better reflect corresponding text descriptions. For cases where Stage-I GAN has generated plausible shapes and colors, Stage-II GAN completes the details. For instance, in the 1st column of Fig. 10, with a satisfactory Stage-I result, Stage-II GAN focuses on drawing the short beak and white color described in the text as well as details for the tail and legs. In all other examples, different degrees of details are added to Stage-II images. In many other cases, Stage-II GAN is able to correct the defects of Stage-I results by processing the text description again. For example, while the Stage-I image in the 5th column has a blue crown rather than the red-brown crown described in the text, the defect is corrected by Stage-II GAN. In some extreme cases (e.g., the 7th column of Fig. 10), even when Stage-I GAN fails to draw a plausible shape, Stage-II GAN is able to generate reasonable objects. We also observe that StackGAN-v1 has the ability to transfer background from Stage-I images and fine-tune them to be more realistic with higher resolution at Stage-II.

Importantly, the StackGAN-v1 does not achieve good results by simply memorizing training samples but by capturing the complex underlying language-image relations. By feeding our generated images and all training images to the Stage-II discriminator D of our StackGAN-v1, their visual features are extracted from the last Conv. layer of D .



Fig. 12. For generated images (column 1), retrieving their nearest training images (columns 2-6) by utilizing Stage-II discriminator of StackGAN-v1 to extract visual features. The L_2 distances between features are calculated for nearest-neighbor retrieval.

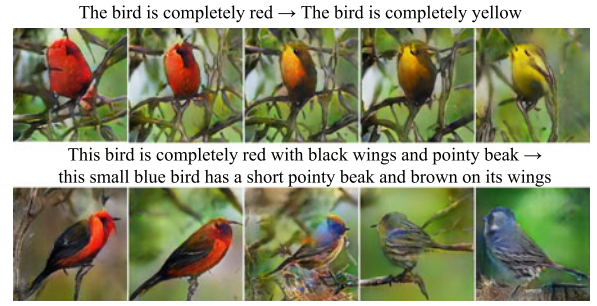


Fig. 13. (Left to right) Images generated by interpolating two sentence embeddings. Gradual appearance changes from the first sentence's meaning to that of the second sentence can be observed. The noise vector z is fixed to be zeros for each row.

And then, we can compute the similarity between two images, based on their visual features. Finally, for each generated image, its nearest neighbors from the training set can be retrieved. By visually inspecting the retrieved images (see Fig. 12), we conclude that the generated images have some similar characteristics with the training samples but are essentially different.

Conditioning Augmentation. We also investigate the efficacy of the proposed Conditioning Augmentation (CA). By removing it from StackGAN-v1 256×256 (denoted as “no CA” in Table 4), the inception score decreases from 3.70 to 3.31. Fig. 11 also shows that 256×256 Stage-I GAN (and StackGAN-v1) with CA can generate birds with different poses and viewpoints from the same text embedding. In contrast, without using CA, samples generated by 256×256 Stage-I GAN collapse to nonsensical images due to the unstable training dynamics of GANs. Consequently, the proposed Conditioning Augmentation helps stabilize the conditional GAN training and improves the diversity of the generated samples because of its ability to encourage robustness to small perturbations along the latent manifold.

Sentence Embedding Interpolation. To further demonstrate that our StackGAN-v1 learns a smooth latent data manifold, we use it to generate images from linearly interpolated sentence embeddings, as shown in Fig. 13. We fix the noise vector z , so the generated image is inferred from the given text description only. Images in the first row are generated by simple sentences made up by us. Those sentences contain only simple color descriptions. The results show that the generated images from interpolated embeddings can accurately reflect color changes and generate plausible bird shapes. The second row illustrates samples generated from more complex sentences, which contain more details on bird appearances. The generated images change their primary color from red to blue, and change the wing color from black to brown.

6.3 The Component Analysis of StackGAN-v2

In this section, we analyze important components of the proposed StackGAN-v2. Table 5 lists models with different settings and their inception scores on the CUB test set. Fig. 14 shows example images generated by different baseline models.

Our baseline models are built by removing or changing a certain component of the StackGAN-v2 model. By approximating the image distribution directly at the 256×256 scale without intermediate branches, the inception scores on CUB

TABLE 5
Inception Scores by Our StackGAN-v2 and Its Baseline Models on CUB test set

Model	branch G_0	branch G_1	branch G_2	JCU	inception score
StackGAN-v2	64×64	128×128	256×256	yes	$4.04 \pm .05$
StackGAN-v2-no-JCU	64×64	128×128	256×256	no	$3.77 \pm .04$
StackGAN-v2- G_2	removed	removed	256×256	yes	$3.49 \pm .04$
StackGAN-v2-3 G_2	removed	removed	three 256×256	yes	$3.22 \pm .02$
StackGAN-v2-all256	256×256	256×256	256×256	yes	$2.89 \pm .02$

“JCU” means using the proposed discriminator that jointly approximates conditional and unconditional distributions.

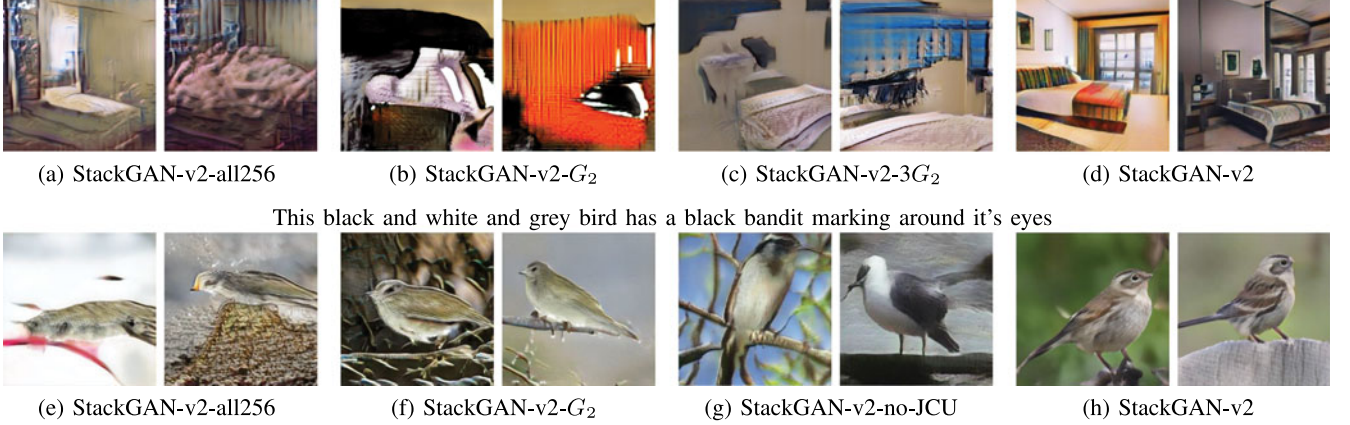


Fig. 14. Example images generated by the StackGAN-v2 and its baseline models on LSUN bedroom (top) and CUB (bottom) datasets.

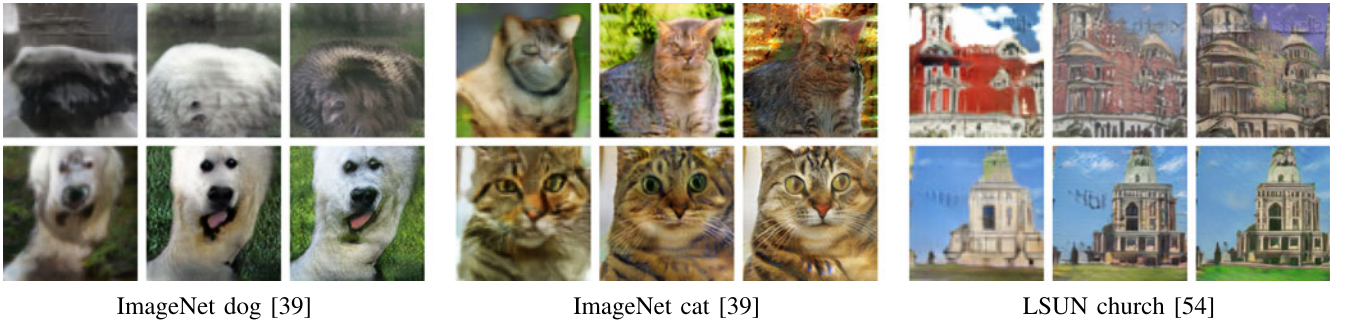


Fig. 15. Example images generated without (top) and with (bottom) the proposed color-consistency regularization for our StackGAN-v2 on ImageNet dog, ImageNet cat and LSUN church datasets. (Left to right) 64×64 , 128×128 , and 256×256 images by G_0 , G_1 , G_2 , respectively.

dramatically decrease from 4.04 to 3.49 for “StackGAN-v2- G_2 ” and to 2.89 for “StackGAN-v2-all256” (See Table 5 and Figs. 14e, 14f). This demonstrates the importance of the multi-scale, multi-stage architecture in StackGAN-v2. Inspired by [10], we also build a baseline model with multiple discriminators at the 256×256 scale, namely “StackGAN-v2-3 G_2 ”. Those discriminators have the same structure but different initializations. However, the results do not show improvement over “StackGAN-v2- G_2 ”. Similar comparisons have also been done for the unconditional task on the LSUN bedroom dataset. As shown in Figs. 14a, 14b, 14c, those baseline models fail to generate realistic images because they suffer from severe mode collapses.

To further demonstrate the effectiveness of jointly approximating conditional and unconditional distributions, “StackGAN-v2-no-JCU” replaces the jointly conditional and unconditional discriminators with the conventional ones, resulting in much lower inception score than that of “StackGAN-v2”. Another baseline model does not use the color-consistency regularization term. Results on various

datasets (see Fig. 15) show that the color-consistency regularization has significant positive effects for the unconditional image synthesis task. Quantitatively, removing the color-consistency regularization decreases the inception score from 9.55 ± 0.11 to 9.02 ± 0.14 on the ImageNet dog dataset. It demonstrates that the additional constraint provided by the color-consistency regularization is able to facilitate multi-distribution approximation and help generators at different branches produce more coherent samples. It is worth mentioning that there is no need to utilize the color-consistency regularization for the text-to-image synthesis task because the text conditioning appears to provide sufficient constraints. Experimentally, adding the color-consistency regularization did not improve the inception score on CUB dataset.

7 CONCLUSIONS

In this paper, Stacked Generative Adversarial Networks, StackGAN-v1 and StackGAN-v2, are proposed to decompose the difficult problem of generating realistic high-

resolution images into more manageable sub-problems. The StackGAN-v1 with Conditioning Augmentation is first proposed for text-to-image synthesis through a novel sketch-refinement process. It succeeds in generating images of 256×256 resolution with photo-realistic details from text descriptions. To further improve the quality of generated samples and stabilize GANs' training, the StackGAN-v2 jointly approximates multiple related distributions, including (1) multi-scale image distributions and (2) jointly conditional and unconditional image distributions. In addition, a color-consistency regularization is proposed to facilitate multi-distribution approximation. Extensive quantitative and qualitative results demonstrate that our proposed methods significantly improve the state of the art in both conditional and unconditional image generation tasks.

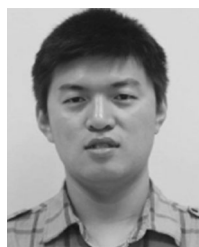
ACKNOWLEDGMENTS

This work is partially supported by the Air Force Office of Scientific Research (AFOSR) under the Dynamic Data-Driven Application Systems Program and National Science Foundation (NSF) 1763523, 1747778, 1733843 and 1703883 Awards; partially supported by the NSF under grant ABI-1661280 and the CNS-1629913; and partially supported by the General Research Fund sponsored by the Research Grants Council of Hong Kong (Nos. CUHK14213616, CUHK14206114, CUHK14205615, CUHK14203015, CUHK14239816, CUHK14219412, CUHK14207814, CUHK14208417, CUHK14202217). Han Zhang and Tao Xu contributed equally to this work.

REFERENCES

- [1] T. Salimans, H. Zhang, A. Radford, and D. N. Metaxas, "Improving GANs using optimal transport," in *Proc. ICLR*, 2018.
- [2] M. Arjovsky and L. Bottou, "Towards principled methods for training generative adversarial networks," in *Proc. ICLR*, 2017.
- [3] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein GAN," in *CoRR*, vol. abs/1701.07877, 2017.
- [4] A. Brock, T. Lim, J. M. Ritchie, and N. Weston, "Neural photo editing with introspective adversarial networks," in *Proc. ICLR*, 2017.
- [5] T. Che, Y. Li, A. P. Jacob, Y. Bengio, and W. Li, "Mode regularized generative adversarial networks," in *Proc. ICLR*, 2017.
- [6] T. Che, Y. Li, R. Zhang, R. D. Hjelm, W. Li, Y. Song, and Y. Bengio, "Maximum-likelihood augmented discrete generative adversarial networks," *J. CoRR*, vol. abs/1702.07983, 2017.
- [7] X. Chen, Y. Duan, R. Houthoofd, J. Schulman, I. Sutskever, and P. Abbeel, "InfoGAN: Interpretable representation learning by information maximizing generative adversarial nets," in *Proc. Conf. Neural Inf. Process. Syst.*, 2016, pp. 2172–2180.
- [8] E. L. Denton, S. Chintala, A. Szlam, and R. Fergus, "Deep generative image models using a laplacian pyramid of adversarial networks," in *Proc. 28th Int. Conf. Neural Inf. Process. Syst.*, 2015, pp. 1486–1494.
- [9] C. Doersch, "Tutorial on variational autoencoders," *J. CoRR*, vol. abs/1606.05908, 2016.
- [10] I. P. Durugkar, I. Gemp, and S. Mahadevan, "Generative multi-adversarial networks," in *Proc. ICLR*, 2017.
- [11] J. Gauthier, "Conditional generative adversarial networks for convolutional face generation," *Tech. Rep.*, 2015.
- [12] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. C. Courville, and Y. Bengio, "Generative adversarial nets," in *Proc. Neural Inf. Process. Syst.*, 2014, pp. 2672–2680.
- [13] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. C. Courville, "Improved training of Wasserstein gans," in *Proc. Conf. Neural Inf. Process. Syst.*, 2017, pp. 5769–5779.
- [14] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [15] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "Gans trained by a two time-scale update rule converge to a local nash equilibrium," in *Proc. Conf. Neural Inf. Process. Syst.*, 2017, pp. 6629–6640.
- [16] X. Huang, Y. Li, O. Poursaeed, J. Hopcroft, and S. Belongie, "Stacked generative adversarial networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 1866–1875.
- [17] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. 32nd Int. Conf. Int. Conf. Mach. Learn.*, 2015, pp. 448–456.
- [18] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 5967–5976.
- [19] T. Karras, T. Aila, S. Laine, and J. Lehtinen, "Progressive growing of gans for improved quality, stability, and variation," in *Proc. ICLR*, 2018.
- [20] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.
- [21] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," in *Proc. ICLR*, 2014.
- [22] A. B. L. Larsen, S. K. Sønderby, H. Larochelle, and O. Winther, "Autoencoding beyond pixels using a learned similarity metric," in *Proc. 33rd Int. Conf. Mach. Learn.*, 2016, pp. 1558–1566.
- [23] C. Ledig, L. Theis, F. Huszar, J. Caballero, A. Aitken, A. Tejani, J. Totz, Z. Wang, and W. Shi, "Photo-realistic single image super-resolution using a generative adversarial network," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 105–114.
- [24] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollr, and C. L. Zitnick, "Microsoft coco: Common objects in context," in *Proc. Eur. Conf. Comput. Vis.*, 2014, pp. 740–755.
- [25] E. Mansimov, E. Parisotto, L. J. Ba, and R. Salakhutdinov, "Generating images from captions with attention," in *Proc. ICLR*, 2016.
- [26] X. Mao, Q. Li, H. Xie, R. Y. K. Lau, Z. Wang, and S. P. Smolley, "Least squares generative adversarial networks," in *Proc. IEEE Conf. Comput. Vis.*, 2017, pp. 2813–2821.
- [27] L. Metz, B. Poole, D. Pfau, and J. Sohl-Dickstein, "Unrolled generative adversarial networks," in *Proc. ICLR*, 2017.
- [28] M. Mirza and S. Osindero, "Conditional generative adversarial nets," *J. CoRR*, vol. abs/1411.1784, 2014.
- [29] A. Nguyen, J. Yosinski, Y. Bengio, A. Dosovitskiy, and J. Clune, "Plug & play generative networks: Conditional iterative generation of images in latent space," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit*, 2017, pp. 3510–3520.
- [30] M.-E. Nilsback and A. Zisserman, "Automated flower classification over a large number of classes," in *Proc. 6th Indian Conf. Comput. Vis. Graph. Image Process.*, 2008, pp. 722–729.
- [31] A. Odena, C. Olah, and J. Shlens, "Conditional image synthesis with auxiliary classifier GANs," in *Proc. 34th Int. Conf. Mach. Learn.*, 2017, pp. 2642–2651.
- [32] A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," in *Proc. ICLR*, 2016.
- [33] S. Reed, Z. Akata, S. Mohan, S. Tenka, B. Schiele, and H. Lee, "Learning what and where to draw," in *Proc. 30th Int. Conf. Neural Inf. Process. Syst.*, 2016, pp. 217–225.
- [34] S. Reed, Z. Akata, B. Schiele, and H. Lee, "Learning deep representations of fine-grained visual descriptions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 49–58.
- [35] S. Reed, Z. Akata, X. Yan, L. Logeswaran, B. Schiele, and H. Lee, "Generative adversarial text-to-image synthesis," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1060–1069.
- [36] S. Reed, A. van den Oord, N. Kalchbrenner, V. Bapst, M. Botvinick, and N. de Freitas, "Generating interpretable images with controllable structure," *Tech. Rep.*, 2016.
- [37] D. J. Rezende, S. Mohamed, and D. Wierstra, "Stochastic backpropagation and approximate inference in deep generative models," in *Proc. Int. Conf. Mach. Learn.*, 2014, pp. 1278–1286.
- [38] D. L. Ruderman, "The statistics of natural images," *Netw.: Comput. Neural Syst.*, vol. 5, no. 4, pp. 517–548, 1994.
- [39] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei, "ImageNet Large Scale Visual Recognition Challenge," *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, 2015.
- [40] T. Salimans, I. J. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen, "Improved techniques for training GANs," in *Proc. 30th Int. Conf. Neural Inf. Process. Syst.*, 2016, pp. 2234–2242.

- [41] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 2818–2826.
- [42] C. K. Snderby, J. Caballero, L. Theis, W. Shi, and F. Huszar, "Amortised map inference for image super-resolution," in *Proc. ICLR*, 2017.
- [43] Y. Taigman, A. Polyak, and L. Wolf, "Unsupervised cross-domain image generation," in *Proc. ICLR*, 2017.
- [44] A. van den Oord, N. Kalchbrenner, and K. Kavukcuoglu, "Pixel recurrent neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1747–1756.
- [45] A. van den Oord, N. Kalchbrenner, O. Vinyals, L. Espeholt, A. Graves, and K. Kavukcuoglu, "Conditional image generation with pixelcnn decoders," in *Proc. 30th Int. Conf. Neural Inf. Process. Syst.*, 2016, pp. 4797–4805.
- [46] L. van der Maaten and G. Hinton, "Visualizing high-dimensional data using t-sne," *J. Mach. Learn. Res.*, vol. 9: 2579–2605, Nov. 2008.
- [47] O. Vinyals, C. Blundell, T. Lillicrap, K. Kavukcuoglu, and D. Wierstra, "Matching networks for one shot learning," in *Proc. 30th Int. Conf. Neural Inf. Process. Syst.*, 2016, pp. 3630–3638.
- [48] C. Vondrick, H. Pirsiavash, and A. Torralba, "Generating videos with scene dynamics," in *Proc. 30th Int. Conf. Neural Inf. Process. Syst.*, 2016, pp. 613–621.
- [49] C. Wah, S. Branson, P. Welinder, P. Perona, and S. Belongie, "The Caltech-UCSD Birds-200–2011 Dataset," California Inst. Technol., Pasadena, CA, USA, Tech. Rep. CNS-TR-2011-001, 2011.
- [50] X. Wang and A. Gupta, "Generative image modeling using style and structure adversarial networks," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 318–335.
- [51] Z. Wang, E. Simoncelli, and A. C. Bovik, "Multi-scale structural similarity for image quality assessment," in *Proc. 37th Signals Syst. Comput.*, 2003, pp. 1398–1402.
- [52] X. Yan, J. Yang, K. Sohn, and H. Lee, "Attribute2image: Conditional image generation from visual attributes," in *Proc. 14th Eur. Conf. Comput. Vis.*, 2016, pp. 776–791.
- [53] J. Yang, A. Kannan, D. Batra, and D. Parikh, "LR-GAN: layered recursive generative adversarial networks for image generation," in *Proc. ICLR*, 2017.
- [54] F. Yu, Y. Zhang, S. Song, A. Seff, and J. Xiao, "LSUN: Construction of a large-scale image dataset using deep learning with humans in the loop," *J. CoRR*, vol. abs/1506.03365, 2015.
- [55] H. Zhang, T. Xu, H. Li, S. Zhang, X. Wang, X. Huang, and D. Metaxas, "StackGAN: Text to photo-realistic image synthesis with stacked generative adversarial networks," in *Proc. IEEE Conf. Comput. Vis.*, 2017, pp. 5908–5916.
- [56] J. Zhao, M. Mathieu, and Y. LeCun, "Energy-based generative adversarial network," in *Proc. ICLR*, 2017.
- [57] J. Zhu, P. Krähenbühl, E. Shechtman, and A. A. Efros, "Generative visual manipulation on the natural image manifold," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 597–613.



Han Zhang received the BS degree in information science from China Agricultural University, Beijing, China, in 2009, and the ME degree in communication and information systems from Beijing University of Posts and Telecommunications, Beijing, China, in 2012. He is currently working toward the PhD degree in the Department of Computer Science, Rutgers University, Piscataway, New Jersey. His current research interests include computer vision, deep learning, and medical image processing.



Tao Xu received the BE degree in agricultural mechanization and automatization from China Agricultural University, Beijing, China, in 2010, and the MS degree in computer science from the Institute of Computing Technology, Chinese Academy of Science, Beijing, China, in 2013. She is currently working toward the PhD degree in the Department of Computer Science and Engineering, Lehigh University, Bethlehem, PA. Her current research interests include deep learning, computer vision, and medical image processing.



Hongsheng Li received the bachelor's degree in automation from the East China University of Science and Technology, and the master's and doctorate degrees in computer science from Lehigh University, Pennsylvania, in 2006, 2010, and 2012, respectively. He is currently with the Department of Electronic Engineering, Chinese University of Hong Kong. His research interests include computer vision, medical image analysis, and machine learning.



Shaoting Zhang received the BE degree from Zhejiang University, in 2005, the MS degree from Shanghai Jiao Tong University, in 2007, and the PhD degree in computer science from Rutgers, in January 2012. His research interests include interface of medical imaging informatics, large-scale visual understanding, and machine learning. He is a senior member of the IEEE.



Xiaogang Wang received the BS degree from the University of Science and Technology of China, in 2001, the MS degree from the Chinese University of Hong Kong, in 2003, and the PhD degree from the Computer Science and Artificial Intelligence Laboratory, Massachusetts Institute of Technology, in 2009. He is currently an associate professor with the Department of Electronic Engineering, The Chinese University of Hong Kong. His research interests include computer vision and machine learning. He is a member of the IEEE.



Xiaolei Huang received the bachelor's degree in computer science from Tsinghua University, China, and the doctorate degree in computer science from Rutgers University–New Brunswick. She is currently an associate professor with the Computer Science and Engineering Department, Lehigh University, Bethlehem, Pennsylvania. Her research interests include computer vision, biomedical image analysis, computer graphics, and machine learning. In these areas she has published articles in journals such as the *IEEE Transactions on Pattern Analysis and Machine Intelligence*, the *MedIA*, the *IEEE Transactions on Medical Imaging*, the *ACM Transactions on Graphics*, and Scientific Reports. She also regularly contributes research papers to conferences such as CVPR, MICCAI, and ICCV. She serves as an associate editor for the *Computer Vision and Image Understanding Journal*. She is a member of the IEEE.



Dimitris N. Metaxas received the BE degree from the National Technical University of Athens Greece, in 1986, the MS degree from the University of Maryland, in 1988, and the PhD degree from the University of Toronto, in 1992. He is a professor with the Computer Science Department, Rutgers University. He is directing the Computational Biomedicine Imaging and Modeling Center (CBIM). He has been conducting research toward the development of formal methods upon which computer vision, computer graphics, and medical imaging can advance synergistically. He is a fellow of the IEEE.

► For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/publications/dlib.